

Seven Sketches in Compositionality: An Invitation to Applied Category Theory

Myxogastria0808

1. 目次

1. 目次	1	4.6. 添字圏と図式	31
2. 圏・対象・射・可換	4	4.7. 関手圏	32
2.1. はじめに	5	5. RDB のスキーマおよびデータを圏に変換	34
2.2. 圏論とは	6	5.1. RDB のスキーマを有向グラフに変換する。	38
2.3. 対象・射	7	5.2. 有向グラフを自由圏に変換する。	41
2.4. 圏 $\mathcal{C}$ の定義	8	5.3. 自由圏に等式制約を加えて得られる有限表示をもつ圏を作成する。	42
2.5. 可換	12	6. RDB のマイグレーション後のデータ整合ができていることを確認する証明	43
3. 関手・自然変換	13	6.1. はじめに	44
3.1. 関手 (共変関手)	14	6.2. 証明の概要	45
3.2. 自然変換	16	6.3. 証明①の方針	46
4. 様々な圏	19	6.4. 証明②の方針	47
4.1. 有向グラフ (これは圏ではない)	20	6.5. 証明①	49
4.2. 自由圏	21		
4.3. 自由圏の例 $\mathbf{Free}(\mathbf{2})$ の場合	22		
4.4. 有限表示をもつ圏	25		
4.5. 圏 $\mathbf{Set}$	28		

1. 目次

6.6. 証明②	52
----------------	----

2. 圏・対象・射・可換

2.1. はじめに

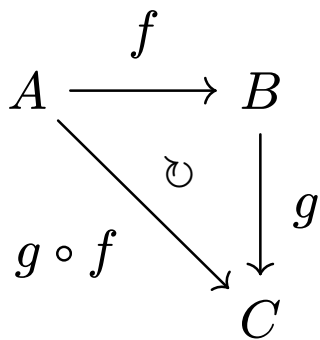
発表を始めるにあたって、数学の世界でよく使われる言い回しを紹介する。
以降の説明の中で、これらの言い回しが出てきた場合は、以下の意味で使われていると理解して欲しい。

- `well-defined`…意味、値、対象が一意に定まる
- `ill-defined`…意味、値、対象が一意に定まらない
- `well-formed`…形式、文法が正しい (式として成立する)
- `self-contained`…自己完結的・それだけで完結している
 - ▶ `self-contained` な説明とは、その説明のみで定義から定理証明までが、包括的に記載されていることを指す。
- `coherent / coherence condition`…複数の構造・操作・対応が互いに整合している
 - ▶ `coherence condition` とは、それらを整合させるために要求される等式や可換条件を指す。

2.2. 圏論とは

- 構造を議論する数学の一分野
- 抽象の数学の一つ

(数学に親しんでいる方にとっては具体的と主張されることもある)



このように、点と矢印のみで構成される世界

2.3. 対象・射



「重要なポイント」

射 = 矢印 という認識をするのではなく、移す元の対象と移した先の対象のタプルに名前を付けたものという認識をすると良い。

(矢印そのものにラベルが付いてる訳ではないという話である。)

2.4. 圏 $\mathcal{C}$ の定義

$\mathcal{C}$ が圏であるためには、以下の 6 つの制約を満たす必要がある。

1. 対象の集まり $\mathcal{Ob}(\mathcal{C})$ を規定
2. $\forall c, d \in \mathcal{Ob}(\mathcal{C})$ に対して、 c から d への射 $\mathcal{C}(c, d)$ を規定
 - 射の集まりを $\mathcal{Hom}(\mathcal{C})$ と規定
3. $\forall c \in \mathcal{Ob}(\mathcal{C})$ に対して、 c 上の恒等射 $\text{id}_c \in \mathcal{C}(c, c)$ を規定
4. $\forall c, d, e \in \mathcal{Ob}(\mathcal{C})$ と射 $f \in \mathcal{C}(c, d), g \in \mathcal{C}(d, e)$ に対して、 f と g の合成射 $f; g \in \mathcal{C}(c, e)$ を規定
 - $c \in \mathcal{Ob}(\mathcal{C})$ と $c \in \mathcal{C}$ を等価な表現とする
 - $f \in \mathcal{C}(c, d)$ と $f : c \rightarrow d$ を等価な表現とする

2.4. 圏 $\mathcal{C}$ の定義

なお、射は以下の要件を満たす必要がある

5. 単位律... $\forall f : c \rightarrow d$ に対して、それを c または d の恒等射と合成しても何も変わらない。すなわち、 $\text{id}_c; f = f$ および $f; \text{id}_d = f$ である。
6. 結合律... $\forall f : c_0 \rightarrow c_1, g : c_1 \rightarrow c_2, h : c_2 \rightarrow c_3$ に対して、 $(f; g); h = f; (g; h)$ が成立する。この合成を単に $f; g; h$ と表現する。

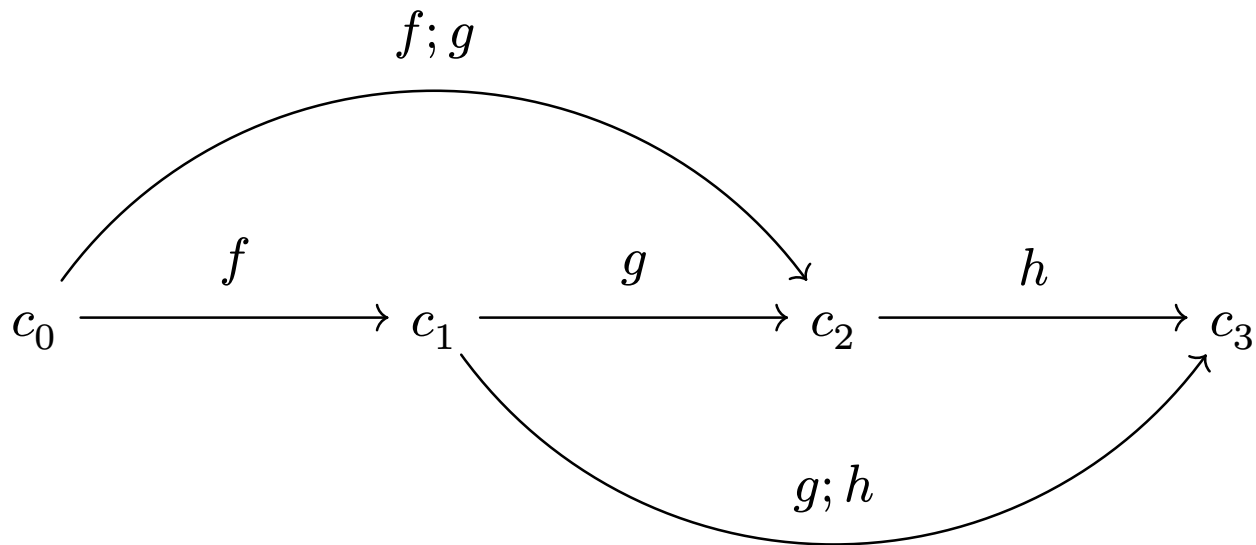
2.4. 圏 $\mathcal{C}$ の定義

単位律の図式は以下のように表現できる。

$$\begin{array}{ccc} \text{id}_c & & \text{id}_d \\ \curvearrowright & \xrightarrow{f} & \curvearrowright \\ c & & d \end{array}$$

2.4. 圏 $\mathcal{C}$ の定義

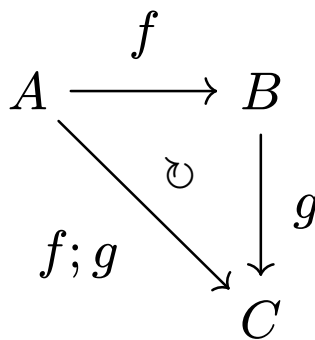
結合律の図式は以下のように表現できる。



2.5. 可換

可換であるとは、図式の中で複数の射が存在する場合に、それらの射を合成した結果が同じになることを指す。

例えば、以下のような三角形の図式が可換であるとは、2つの射 f, g とそれらの合成射 $f; g$ が等しいことを意味する。



なお、以降の図式では、 $\circ$ の記号は省略することが多い。

3. 関手・自然変換

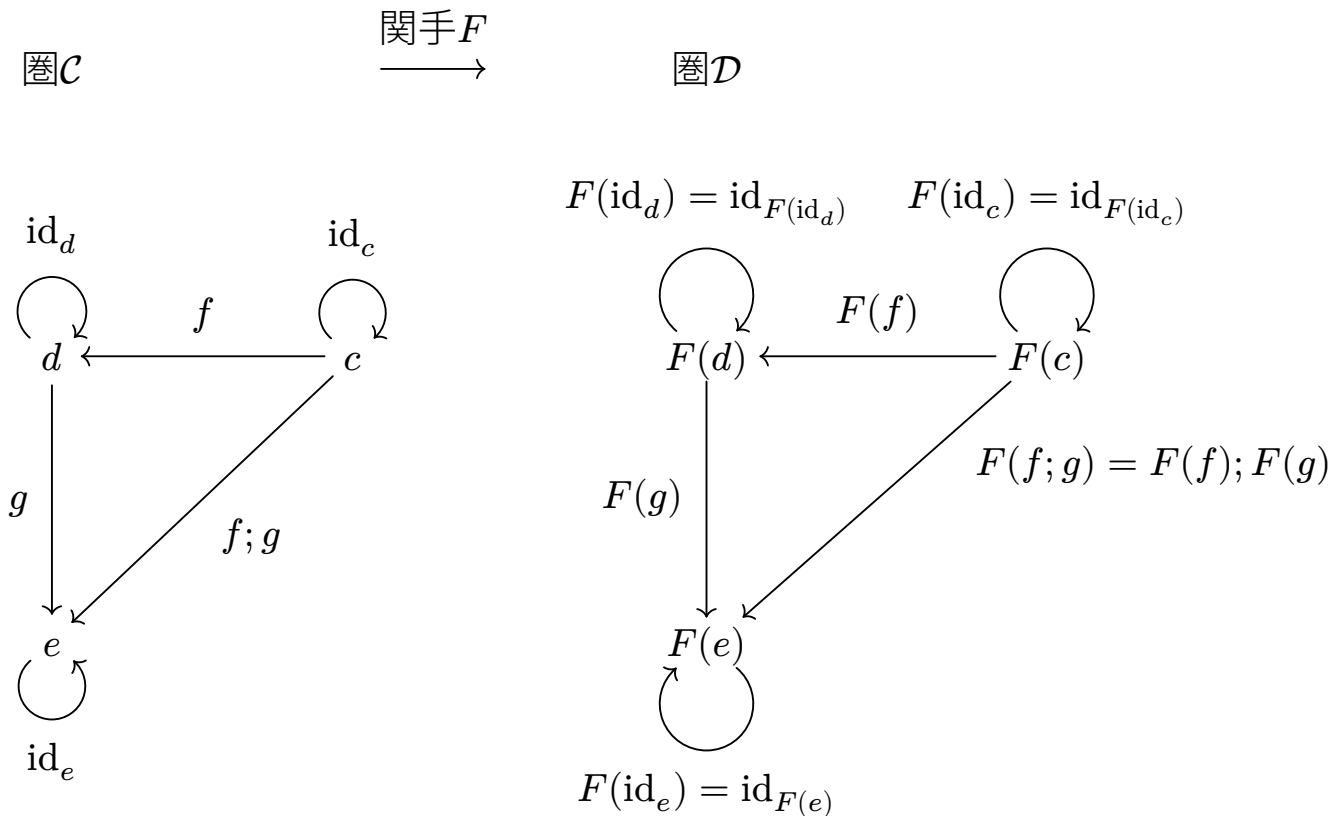
3.1. 関手 (共変関手)

複数の関手

- 対象について
 - ▶ 共変関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ は、圏 $\mathcal{C}$ の任意の対象 c を圏 $\mathcal{D}$ のある対象 $F(c)$ に写す。
- 射について
 - ▶ 共変関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ は、圏 $\mathcal{C}$ の任意の射 $f : c \rightarrow d$ を圏 $\mathcal{D}$ のある射 $F(f) : F(c) \rightarrow F(d)$ に写し、以下を満たす。
 1. 恒等射の保存: $F(\text{id}_c) = \text{id}_{F(c)}$
 2. 射の合成の保存: $F(f; g) = F(f); F(g)$

3.1. 関手 (共変関手)

可換図式で表現すると、以下のようなになる。



3.2. 自然変換

圏 $\mathcal{C}$ と圏 $\mathcal{D}$ について $F, G : \mathcal{C} \rightarrow \mathcal{D}$ を関手とする。自然変換 $\alpha : F \Rightarrow G$ を規定するためには、以下の2つを満たす必要がある。

- コンポーネントの規定
 - ▶ 圏 $\mathcal{C}$ の任意の対象 c に対して、圏 $\mathcal{D}$ の射 $\alpha_c : F(c) \rightarrow G(c)$ を規定する。
 - ▶ このときの射 α_c を自然変換 α の c コンポーネントと呼ぶ。
- 自然性条件
 - ▶ 任意のコンポーネント α_c と α_d と圏 $\mathcal{C}$ の任意の射 $f : c \rightarrow d$ に対して、次スライドの図式が可換であることを要求する。
 - ▶ 式で記述すると、 $\alpha_c; G(f) = F(f); \alpha_d$ である。

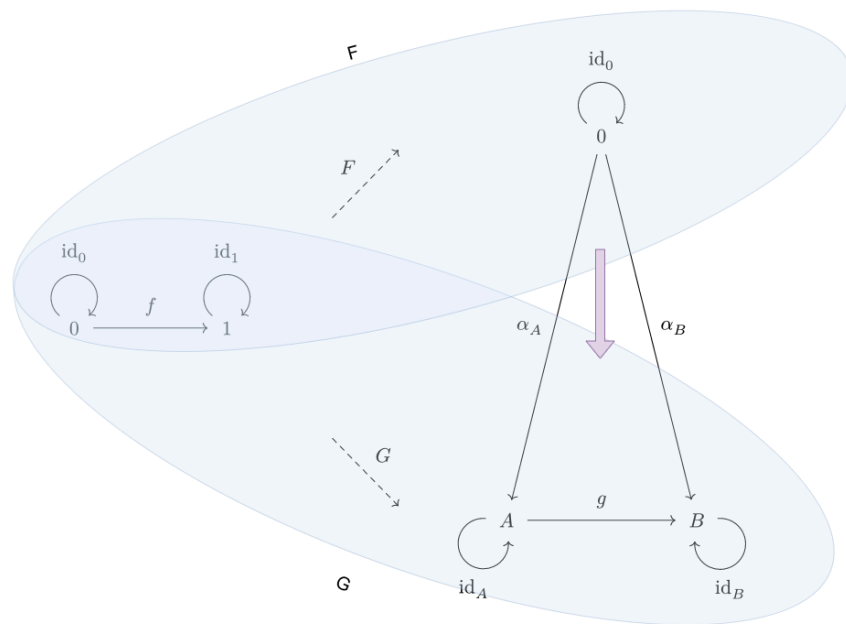
3.2. 自然変換

以下の図式が可換であることによって、自然性条件を満たすことを表現できる。

$$\begin{array}{ccc} F(c) & \xrightarrow{\alpha_c} & G(c) \\ F(f) \downarrow & \circlearrowleft & \downarrow G(f) \\ F(d) & \xrightarrow{\alpha_d} & G(d) \end{array}$$

3.2. 自然変換

青色の円を関手、紫色の矢印を自然変換というイメージを持つと理解しやすい。
(詳細は、関手圏のセクションで説明する。)



4. 様々な圏

4.1. 有向グラフ (これは圏ではない)

$G = (\text{Vertex}, \text{Arrow}, s, t)$ で表現される。

- Vertex…頂点を要素にもつ集合
 - Arrow…辺を要素にもつ集合
 - s …始点関数
 - t …終点関数
 - ▶ 2つの関数 $s, t : \text{Arrow} \rightarrow \text{Vertex}$ は、以下のように定義される。
 - ▶ $s(a) = v$ および $t(a) = w$ であるような $a \in \text{Arrow}$ が与えられたとき、 a は v から w への辺である。この有向グラフは、 $v \xrightarrow{a} w$ のような図式で表現できる。
 - G の道 (path) …それぞれの辺の終点が次の辺の始点であるような辺の列。
 - ▶ 道には、 G の辺 $a \in A$ にすぎない長さ1の列や、同じ頂点を始点および終点として、どの辺もたどらない長さ0の列も含まれる。
 - 事実: 任意の有向グラフから擬順序が得られる。
-

4.2. 自由圏

任意の有向グラフ $G = (\text{Vertex}, \text{Arrow}, s, t)$ に対して、 G 上の自由圏 $\mathbf{Free}(G)$ を定義する。

その圏の対象は頂点 Vertex であり、射は G の道である。対象 c 上の恒等射は単に c における自明な道である。合成は、道の連結によって与えられる。

有向グラフをそのまま圏に変換することができると理解しておけば十分である。

4.3. 自由圏の例 $\mathcal{Free}(\mathbf{2})$ の場合

有向グラフ $c_1 \xrightarrow{f_1} c_2$ によって生成される自由圏 $\mathcal{Free}(\mathbf{2})$ を以下のように定義する。

$$\mathcal{Free}(\mathbf{2}) := \mathcal{Free}\left(v_1 \xrightarrow{f_1} v_2\right)$$

圏であるためには、以下の 6 つの制約を満たす必要があるので、満たしているかどうかを確認する。

1. 対象の集まり $\mathcal{Ob}(\mathcal{C})$ を規定
 - $v_1, v_2 \in \mathcal{Ob}(\mathcal{Free}(\mathbf{2}))$ とする。
2. $\forall c, d \in \mathcal{Ob}(\mathcal{C})$ に対して、 c から d への射 $\mathcal{C}(c, d)$ を規定
 - $\text{id}_{v_1} : v_1 \rightarrow v_1, \text{id}_{v_2} : v_2 \rightarrow v_2, f_1 : v_1 \rightarrow v_2$ とする。

4.3. 自由圏の例 $\mathcal{Free}(\mathbf{2})$ の場合

3. $\forall c \in \mathcal{Ob}(\mathcal{C})$ に対して、 c 上の恒等射 $\text{id}_c \in \mathcal{C}(c, c)$ を規定
 - $\forall v \in \mathcal{Ob}(\mathcal{Free}(\mathbf{2}))$ に対して、 v 上の恒等射が存在する。
 - $\text{id}_{v_1} : v_1 \rightarrow v_1, \text{id}_{v_2} : v_2 \rightarrow v_2$
4. $\forall c, d, e \in \mathcal{Ob}(\mathcal{C})$ と射 $f \in \mathcal{C}(c, d), g \in \mathcal{C}(d, e)$ に対して、 f と g の合成射 $f; g \in \mathcal{C}(c, e)$ を規定
 - 合成は道の結合より与えられる。
 - e.g. $\text{id}_{v_1}; f_1 = f_1, \text{id}_{v_2}; \text{id}_{v_2} = \text{id}_{v_2}$

4.3. 自由圏の例 $\mathcal{Free}(2)$ の場合

なお、射は以下の要件を満たす必要がある

5. 単位律... $\forall f : c \rightarrow d$ に対して、それを c または d の恒等射と合成しても何も変わらない。すなわち、 $\text{id}_c; f = f$ および $f; \text{id}_d = f$ である。
 - $\text{id}_{v_1}; f_1 = f_1, f_1; \text{id}_{v_2} = f_1$ が成り立つ。
6. 結合律... $\forall f : c_0 \rightarrow c_1, g : c_1 \rightarrow c_2, h : c_2 \rightarrow c_3$ に対して、 $(f; g); h = f; (g; h)$ が成立する。この合成を単に $f; g; h$ と表現する。
 - $\text{id}_{v_1} : v_1 \rightarrow v_1, f_1 : v_1 \rightarrow v_2, \text{id}_{v_2} : v_2 \rightarrow v_2$ に対して、等式 $(\text{id}_{v_1}; f_1); \text{id}_{v_2} = \text{id}_{v_1}; (f_1; \text{id}_{v_2})$ が成り立つ。

従って、 $\mathcal{Free}\left(v_1 \xrightarrow{f_1} v_2\right)$ は圏である。

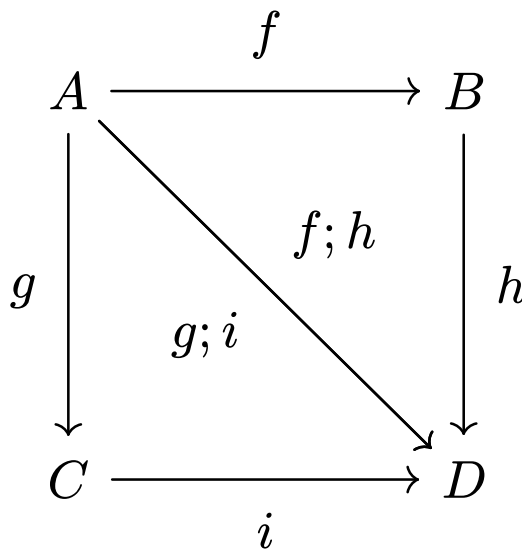
4.4. 有限表示をもつ圏

任意のグラフ G に対して、 G 上の自由圏がある。この圏について、始点 p と終点 q が等しい $f : p \rightarrow q$ と $g : p \rightarrow q$ があるとき、これらは平行と呼ばれ、等式として結ぶこと ($f = g$) が許される。この等式を伴う有限グラフは、圏に対する有限表示とよばれる。そして、その結果の圏を有限表示をもつ圏と定義する。

4.4. 有限表示をもつ圏

自由圏

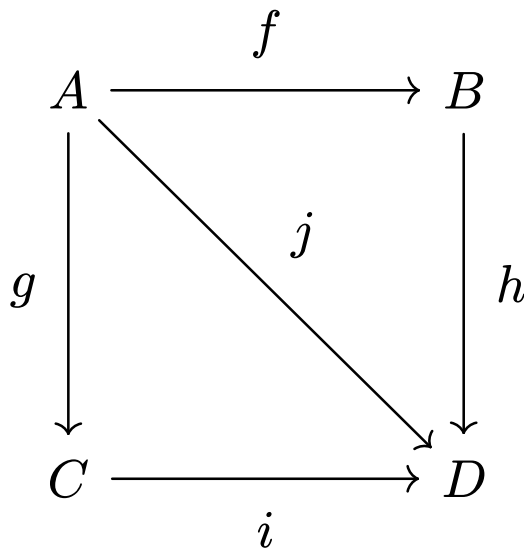
自由圏 (有限表示をもたない圏) は、例えば以下の図式のように $f; h$ と $g; i$ が等しい有限表示をもたない圏である。従って、射の数は10個 ($f, g, h, i, g; i, f; h, \text{id}_A, \text{id}_B, \text{id}_C, \text{id}_D$) となる。



4.4. 有限表示をもつ圏

有限表示をもつ圏

有限表示をもつ圏は、例えば以下の図式のように $f; h$ と $g; i$ が等しい有限表示をもつ圏である。従って、射の数は9個 ($f, g, h, i, j, \text{id}_A, \text{id}_B, \text{id}_C, \text{id}_D$) となる。



$$f; h = g; i = j$$

4.5. 圏 Set

圏 Set は、集合と関数をそれぞれ対象と射に対応させた圏である。

圏であるためには、以下の 6 つの制約を満たす必要があるので、満たしているかどうかを確認する。

1. 対象の集まり $\mathcal{Ob}(\mathcal{C})$ を規定
 - $\mathcal{Ob}(Set)$ は、すべての集合の集まりである。
 2. $\forall c, d \in \mathcal{Ob}(\mathcal{C})$ に対して、 c から d への射 $\mathcal{C}(c, d)$ を規定
 - 集合 S, T について、 $Set(S, T) = \{f : S \rightarrow T \mid f \text{ は関数}\}$ である。
 3. $\forall c \in \mathcal{Ob}(\mathcal{C})$ に対して、 c 上の恒等射 $\text{id}_c \in \mathcal{C}(c, c)$ を規定
 - 集合 S における恒等射は、それぞれの $s \in S$ に対して $\text{id}_{S(s)} := s$ で与えられる関数 $\text{id}_S : S \rightarrow S$ である。
-

4.5. 圏 Set

4. $\forall c, d, e \in \mathcal{Ob}(\mathcal{C})$ と射 $f \in \mathcal{C}(c, d), g \in \mathcal{C}(d, e)$ に対して、 f と g の合成射 $f; g \in \mathcal{C}(c, e)$ を規定

- 与えられた関数 $f : S \rightarrow T$ と $g : T \rightarrow U$ に対して、それらの合成は $(f; g)(s) := g(f(s))$ で与えられる関数 $f; g : S \rightarrow U$ である。

なお、射は次の 2 つの要件を満たす必要がある。

5. 単位律... $\forall f : c \rightarrow d$ に対して、それを c または d の恒等射と合成しても何も変わらない。すなわち、 $\text{id}_c; f = f$ および $f; \text{id}_d = f$ である。

- 関数 $f : S \rightarrow T$ について、 $\text{id}_S : \text{id}_{S(s)} = s$ (ただし、任意の $s \in S$) との合成関数 $(\text{id}_S; f)(s) = f(\text{id}_{S(s)}) = f(s)$ である。また、 $\text{id}_T : \text{id}_{T(t)} = t$ (ただし、任意の $t \in T$) との合成関数 $(f; \text{id}_T)(s) = \text{id}_{T(f(s))} = f(s)$ である。従って、単位律の条件を満たす。
-

4. 様々な圏

6. 結合律... $\forall f : c_0 \rightarrow c_1, g : c_1 \rightarrow c_2, h : c_2 \rightarrow c_3$ に対して、 $(f; g); h = f; (g; h)$ が成立する。この合成を単に $f; g; h$ と表現する。

- $f : S \rightarrow T, g : T \rightarrow U, h : U \rightarrow V$ について、 $((f; g); h)(s) = h((f; g)(s)) = h(g(f(s)))$ (ただし、任意の $s \in S$) である。また、 $(f; (g; h))(s) = (g; h)(f(s)) = h(g(f(s)))$ (ただし、任意の $s \in S$) である。任意の $s \in S$ で値が等しいので、関数も等しい。従って、結合律の条件を満たす。

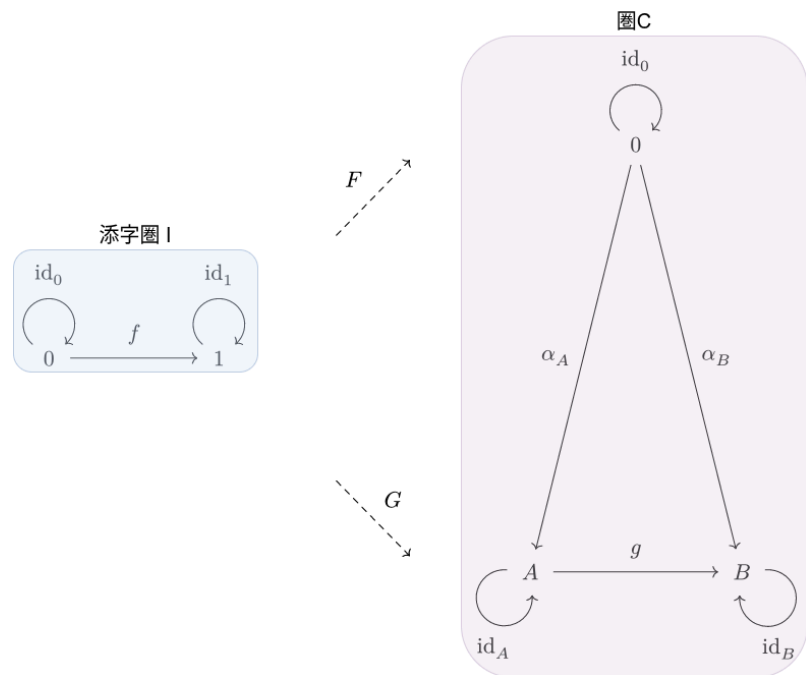
なお、圏 $\mathbf{Set}$ の対象は無限であるが、対象が有限集合で、射がそれらの間の関数であるような圏 $\mathbf{FinSet}$ も定義できる。この圏は圏 $\mathbf{Set}$ に密接に関係している。

4.6. 添字圏と図式

圏 $\mathcal{C}$ 上の $\mathcal{I}$ -型の図式とは、共変関手 $F : \mathcal{I} \rightarrow \mathcal{C}$ や $G : \mathcal{I} \rightarrow \mathcal{C}$ のこと。

この圏 $\mathcal{I}$ のことをこれらの図式の添字圏とよぶ。

(以下の図式には、コンポーネントも存在するが、特に自然変換である必要はない。)



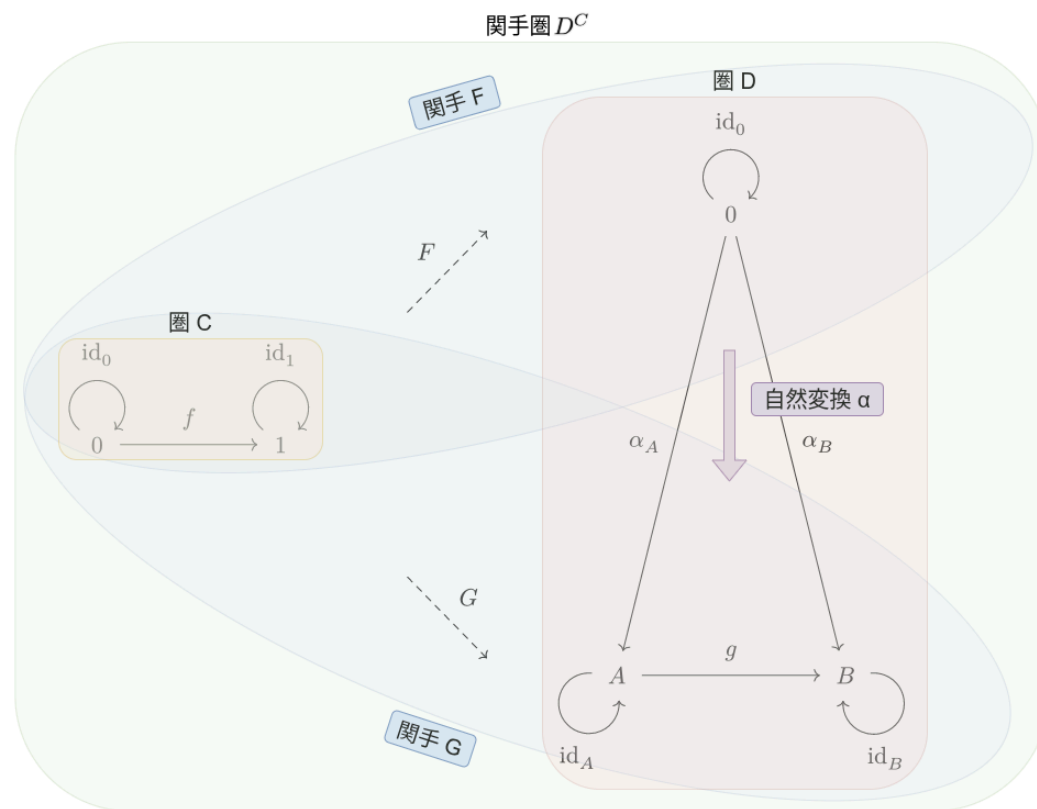
4.7. 関手圏

圏 $\mathcal{C}$ から圏 $\mathcal{D}$ への関手圏 $\mathcal{D}^{\mathcal{C}}$ とは、圏 $\mathcal{C}$ から圏 $\mathcal{D}$ への共変関手を対象とし、それらの間の自然変換を射とする圏のこと。

例えば、圏 $\mathcal{C}$ から圏 $\mathcal{D}$ への関手 $F, G : \mathcal{C} \rightarrow \mathcal{D}$ と、自然変換 $\alpha : F \Rightarrow G$ があるとき、 F, G は関手圏 $\mathcal{D}^{\mathcal{C}}$ の対象であり、 α は射である。

この例を図式で表現すると、次スライドのようになる。

4.7. 関手圏



5. RDB のスキーマおよびデータを圏に変換

RDB のスキーマを圏へ変換する方針は、以下の通りである。

1. RDB のスキーマを有向グラフに変換する。
2. 有向グラフを自由圏に変換する。
3. 自由圏に等式制約を加えて得られる有限表示をもつ圏を作成する。

さらに、RDB のデータを圏に変換するには、以下のステップを踏む。

4. 始域の圏に対して、その圏の元になったスキーマに具体的に与えられていたデータを与える。
 - この操作は、集合値関手 (特に、この場合はデータベース・インスタンス)
 $I : \mathcal{C} \rightarrow \mathbf{Set}$ を作成することが目的である。
-

以下のスキーマを例にして、RDB のスキーマを圏に変換する方法を説明する。

スキーマ C

Employee テーブル

id (primary key)	manager (primary key を内部参照)	dept (foreign key)	description
------------------	-----------------------------	--------------------	-------------

Department テーブル

id (primary key)	note
------------------	------

スキーマ C に関する業務上の制約として、「社員の上司は、その社員と同じ部署に所属している」というものを考える。

スキーマ D

Staff テーブル

id (primary key)	supervisor (primary key を 内部参照)	team (foreign key)	profile (foreign key)
------------------	------------------------------------	--------------------	-----------------------

Profile テーブル

id (primary key)	bio
------------------	-----

Team テーブル

id (primary key)	memo
------------------	------

5.1. RDB のスキーマを有向グラフに変換する。

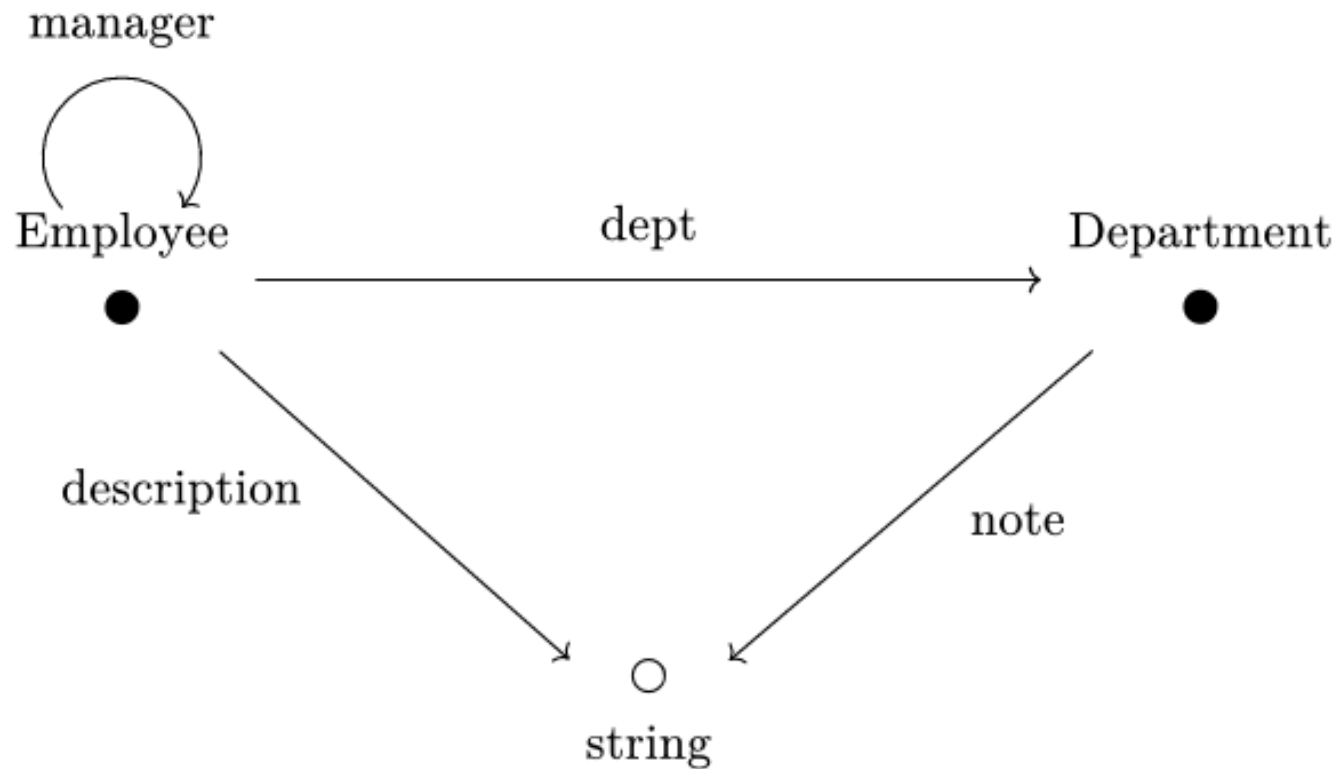
証明の対象とする 2 つのスキーマの有向グラフは、次以降のスライドの通り。

以下に有向グラフの凡例と制約を示す。

- [凡例] Table の id を参照している列については ●、それ以外を参照している列については ○ へ向かう辺として各列の型を表現している。
- [制約] 各列には必ずただ一つの型が与えられなければならない。

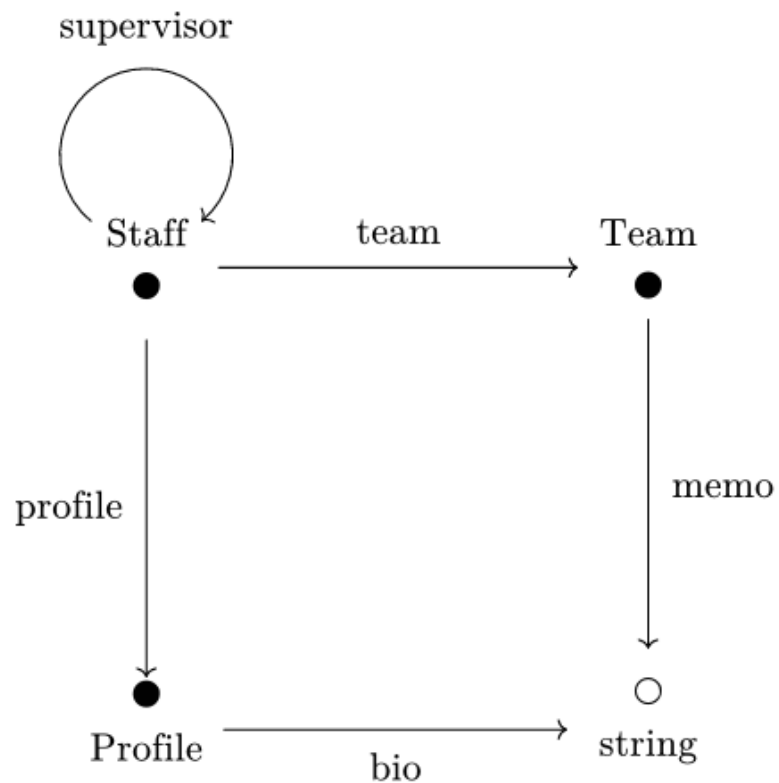
5.1. RDB のスキーマを有向グラフに変換する。

有向グラフ C



5.1. RDB のスキーマを有向グラフに変換する。

有向グラフ D



5.2. 有向グラフを自由圏に変換する。

4.2 のスライドで示した通り、有向グラフ C, D は、自由圏に変換することができる。従って、有向グラフ C, D は、それぞれ自由圏 $\mathcal{C}, \mathcal{D}$ に変換することができる。

有向グラフ $\xrightarrow{\text{変換}}$ 自由圏

5.3. 自由圏に等式制約を加えて得られる有限表示をもつ圏を作成する。

スキーマ C に存在する等式制約「社員の上司は、その社員と同じ部署に所属している」を自由圏 C に加えることによって、有限表示をもつ圏 C を作成する。

等式制約の表現方法は、以下の通りである。

- 表現方法は、通過したノードとエッジを $.$ で連結して記述する。
- e.g. $N_1 \xrightarrow{E_1} N_2$ (N_1 から出発して E_1 を経由し、 N_2 に到達した場合): $N_1.E_1$

圏 C の有限表示は、以下の通りである。

- $\text{Employee.manager.dept} = \text{Employee.dept}$ (圏 C における等式制約)

圏 C の有限表示の制約を圏 D に写した際の有限表示は、以下の通りである。

- $\text{Staff.supervisor.team} = \text{Staff.team}$ (圏 D における等式制約)

以上で、次スライド以降で行う証明の準備ができた。

6. RDB のマイグレーション後のデータ整合ができていることを確認する証明

6.1. はじめに

本来であれば、正しくデータ移行が「行われる」ことを保証すべきであるが、そのためには、Kan 拡張についての理解をする必要がある。そのため、以下の証明は全てデータ移行が「行われた後」のスキーマ整合またはデータ整合に関する確認のみできている。従って、実際の操作はしていない。

6.2. 証明の概要

今回は、次に示す 2 種類の証明を行う。

証明①

スキーマレベル (各列の型のレベル) におけるマイグレーションにおいて、移行元スキーマの各型・各列・各等式制約に、移行先スキーマ上の対応先が存在することを示す。

証明②

データ (インスタンス) レベルにおけるマイグレーションにおいて、移行後のデータベース・インスタンス J は、移行元のデータベース・インスタンス I を関手 F に沿って整合的に受け止めていることを示す。

(データベース・インスタンスについては、次以降のスライドで説明する。)

次スライドでは、各証明の方針を説明する。

6.3. 証明①の方針

1. RDB のスキーマを有向グラフに変換する。
2. 有向グラフを自由圏に変換する。
3. 自由圏に等式制約を加えて得られる有限表示をもつ圏を作成する。
4. 始域の圏 $\mathcal{C}$ と終域の圏 $\mathcal{D}$ を用意し、始域の圏から終域の圏に向かって関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ を用意する。
5. 4.で目的の関手ができたので、スキーマレベル (各列の型のレベル) におけるマイグレーションにおいて、移行元スキーマの各型・各列・各等式制約に、移行先スキーマ上の対応先が存在することが示せる。

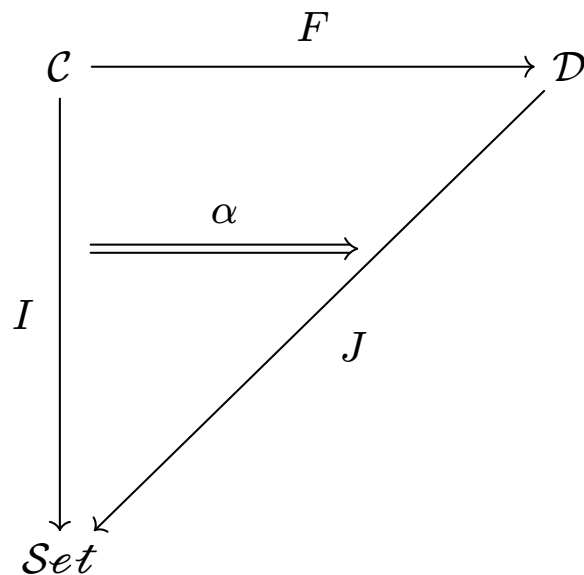
ステップ3までは、前述の通りである。

6.4. 証明②の方針

1. RDB のスキーマを有向グラフに変換する。
 2. 有向グラフを自由圏に変換する。
 3. 自由圏に等式制約を加えて得られる有限表示をもつ圏を作成する。
 4. 始域の圏 $\mathcal{C}$ に対して、その圏の元になったスキーマに具体的に与えられていたデータを与える。すると、圏 Set を用いて $I : \mathcal{C} \rightarrow Set$ という集合値関手を作成できる。この集合値関手をデータベース・インスタンス (スキーマを実際のデータへ結ぶもの) とする。同様に、終域の圏 $\mathcal{D}$ についてのデータベース・インスタンス $J : \mathcal{D} \rightarrow Set$ も用意する。(データベース・インスタンスは、活躍する圏論で実際に定義されている用語である。)
 5. 始域の圏 $\mathcal{C}$ と終域の圏 $\mathcal{D}$ を用意し、始域の圏から終域の圏に向かって関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ を用意する。
-

6.4. 証明②の方針

6. 以下のような図式の自然変換 $\alpha : I \Rightarrow F; J$ を用意する。



7. 6.で目的の自然変換ができたので、自然変換 α の存在により、移行後のインスタンス J は、移行元インスタンス I を関手 F に沿って整合的に受け止めていることが示された。

6.5. 証明①

ステップ 3 までは、前述の通りである。

従って、ステップ 4 から証明の説明を行う。

4. 始域の圏 $\mathcal{C}$ と終域の圏 $\mathcal{D}$ を用意し、関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ を用意する。

→ この関手 F の存在を示せば、スキーマ C からスキーマ D にスキーマレベルでデータ整合したことを示せたことになる。

関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ の存在を示す。

- 対象について
 - ▶ $F(\text{Employee}) = \text{Staff}$
 - ▶ $F(\text{Department}) = \text{Team}$
 - ▶ $F(\text{string}) = \text{string}$
-

6.5. 証明①

- 射について (恒等射は恒等射に写すことを前提としている。)
 - ▶ $F(\text{manager}) = \text{supervisor}$
 - ▶ $F(\text{dept}) = \text{team}$
 - ▶ $F(\text{description}) = \text{profile; bio}$
 - ▶ $F(\text{note}) = \text{memo}$

従って、関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ が存在する。

6.5. 証明①

- 等式制約に違反していなかの確認

等式制約の圏 $\mathcal{C}$ と圏 $\mathcal{D}$ の対応は以下の通り。

$$F(\text{Employee.manager.dept}) = F(\text{Employee.dept})$$

$$\text{Staff.supervisor.team} = \text{Staff.team}$$

従って、3. で示した圏 $\mathcal{D}$ の等式制約を満たす。

よって、スキーマレベル (各列の型のレベル) におけるマイグレーションにおいて、移行元スキーマの各型・各列・各等式制約に、移行先スキーマ上の対応先が存在することを示せた。

6.6. 証明②

ステップ 3 までは、前述の通りである。

従って、ステップ 4 から証明の説明を行う。

4. 始域の圏 $\mathcal{C}$ に対して、その圏の元になったスキーマに具体的に与えられていたデータを与える。すると、圏 $\mathbf{Set}$ を用いて $I : \mathcal{C} \rightarrow \mathbf{Set}$ という集合値関手を作成できる。この集合値関手をデータベース・インスタンス (スキーマを実際のデータへ結ぶもの) とする。同様に、移行先の圏 $\mathcal{D}$ についてのデータベース・インスタンス $J : \mathcal{D} \rightarrow \mathbf{Set}$ も用意する。(データベース・インスタンスは、活躍する圏論で実際に定義されている用語である。)

→ 圏 $\mathcal{C}, \mathcal{D}$ に対して、それぞれデータベース・インスタンス $I : \mathcal{C} \rightarrow \mathbf{Set}, J : \mathcal{D} \rightarrow \mathbf{Set}$ を用意する。

6.6. 証明②

スキーマ*C*には、具体的に以下のデータが入っているとする。

Employee テーブル

id (primary key)	manager (primary key を内部参照)	dept (foreign key)	description
e_1	e_1	d_1	LeadEngineer
e_2	e_1	d_1	BackendEngineer
e_3	e_3	d_2	Accountant

Department テーブル

id (primary key)	note
d_1	EngineeringTeam
d_2	AccountantTeam

6.6. 証明②

従って、データベース・インスタンス I は、以下のように表せる。

- $I(\text{Employee}) = \{e_1, e_2, e_3\}$
(Employee := Employee テーブルの id)
 - $I(\text{Employee.mamanger}) = \{e_1 \mapsto e_1, e_2 \mapsto e_1, e_3 \mapsto e_3\}$
(Employee.manager := Employee テーブルの manager)
 - $I(\text{Employee.dept}) = \{e_1 \mapsto d_1, e_2 \mapsto d_1, e_3 \mapsto d_2\}$
(Employee.dept := Employee テーブルの dept)
 - $I(\text{Employee.description}) = \{e_1 \mapsto \text{"LeadEngineer"}, e_2 \mapsto \text{"BackendEngineer"}, e_3 \mapsto \text{"Accountant"}\}$
(Employee.description := Employee テーブルの description)
-

6.6. 証明②

- $I(\text{Department}) = \{d_1, d_2\}$
(Department := Department テーブルの id)
 - $I(\text{Department.note}) = \{d_1 \mapsto \text{"EngineeringTeam"}, d_2 \mapsto \text{"AccountantTeam"}\}$
(Department.note := Department テーブルの note)
-

6.6. 証明②

スキーマ D には、具体的に以下のデータが入っているとする。

Staff テーブル

id (primary key)	supervisor (primary key を内部参照)	team (foreign key)	profile (foreign key)
e_1	e_1	d_1	p_1
e_2	e_1	d_1	p_2
e_3	e_3	d_2	p_3

Profile テーブル

id (primary key)	bio
p_1	LeadEngineer
p_2	BackendEngineer

6. RDB のマイグレーション後のデータ整合ができていることを確認する証明

p_3	Accountant
-------	------------

Team テーブル

id (primary key)	memo
d_1	EngineeringTeam
d_2	AccountantTeam

6.6. 証明②

従って、データベース・インスタンス I は、以下のように表せる。

- $I(\text{Employee}) = \{e_1, e_2, e_3\}$
(Employee := Employee テーブルの id)
 - $I(\text{Employee.mamanger}) = \{e_1 \mapsto e_1, e_2 \mapsto e_1, e_3 \mapsto e_3\}$
(Employee.manager := Employee テーブルの manager)
 - $I(\text{Employee.dept}) = \{e_1 \mapsto d_1, e_2 \mapsto d_1, e_3 \mapsto d_2\}$
(Employee.dept := Employee テーブルの dept)
 - $I(\text{Employee.description}) = \{e_1 \mapsto \text{"LeadEngineer"}, e_2 \mapsto \text{"BackendEngineer"}, e_3 \mapsto \text{"Accountant"}\}$
(Employee.description := Employee テーブルの description)
-

6.6. 証明②

- $I(\text{Department}) = \{d_1, d_2\}$
(Department := Department テーブルの id)
 - $I(\text{Department.note}) = \{d_1 \mapsto \text{"EngineeringTeam"}, d_2 \mapsto \text{"AccountantTeam"}\}$
(Department.note := Department テーブルの note)
-

6.6. 証明②

5. 始域の圏 $\mathcal{C}$ と終域の圏 $\mathcal{D}$ を用意し、関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ を用意する。

→ 関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ を作成する。(証明①と全く同じ関手でよい。)

関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ の存在を示す。

- 対象について
 - ▶ $F(\text{Employee}) = \text{Staff}$
 - ▶ $F(\text{Department}) = \text{Team}$
 - ▶ $F(\text{string}) = \text{string}$
-

6.6. 証明②

- 射について (恒等射は恒等射に写すことを前提としている。)
 - ▶ $F(\text{manager}) = \text{supervisor}$
 - ▶ $F(\text{dept}) = \text{team}$
 - ▶ $F(\text{description}) = \text{profile; bio}$
 - ▶ $F(\text{note}) = \text{memo}$

従って、関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ が存在する。

6.6. 証明②

- 等式制約に違反していなかの確認 (証明①と同じ確認をすればよい。)

等式制約の圏 $\mathcal{C}$ と圏 $\mathcal{D}$ の対応は以下の通り。

$$F(\text{Employee.manager.dept}) = F(\text{Employee.dept})$$

$$\text{Staff.supervisor.team} = \text{Staff.team}$$

従って、3. で示した圏 $\mathcal{D}$ の等式制約を満たす。

6.6. 証明②

6. 以下のような図式の自然変換 $\alpha : I \Rightarrow F; J$ を用意する。

→ 5. までで用意した関手を用いて、自然変換 $\alpha : I \Rightarrow F; J$ を作成する。

コンポーネントは、圏 $\mathcal{C}$ の各対象ごとに存在する。従って、以下の 3 つがコンポーネントとなる。

$$\alpha_{\text{Employee}} = I(\text{Employee}) \rightarrow J(F(\text{Employee}))$$

$$\alpha_{\text{Department}} = I(\text{Department}) \rightarrow J(F(\text{Department}))$$

$$\alpha_{\text{string}} = I(\text{string}) \rightarrow J(F(\text{string}))$$

6.6. 証明②

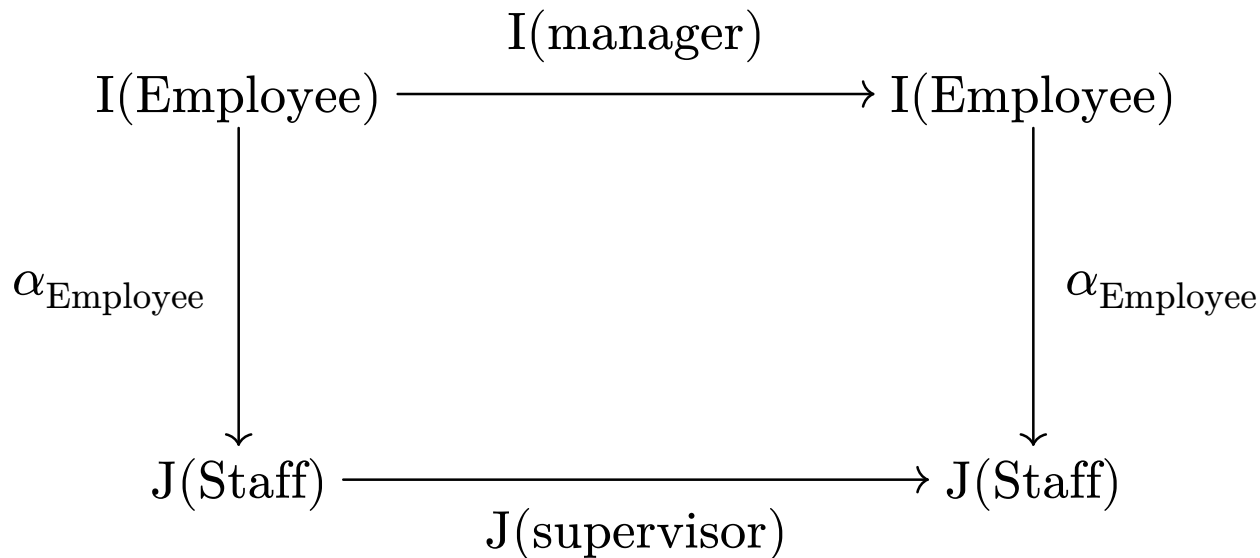
圏 $\mathcal{C}$ には、射が 4 つ存在するので、それぞれの射について自然変換 $\alpha : I \Rightarrow F; J$ の自然条件が成立すれば、自然変換 α が存在することを示せる。なので、それを次以降のスライドで示す。

6.6. 証明②

- 射 $\text{manager} : \text{Employee} \rightarrow \text{Employee}$ に対する自然条件

以下が可換図式である。

このように、自然条件 $I(\text{manager}); \alpha_{\text{Employee}} = \alpha_{\text{Employee}}; J(\text{supervisor})$ が成立していることがわかる。

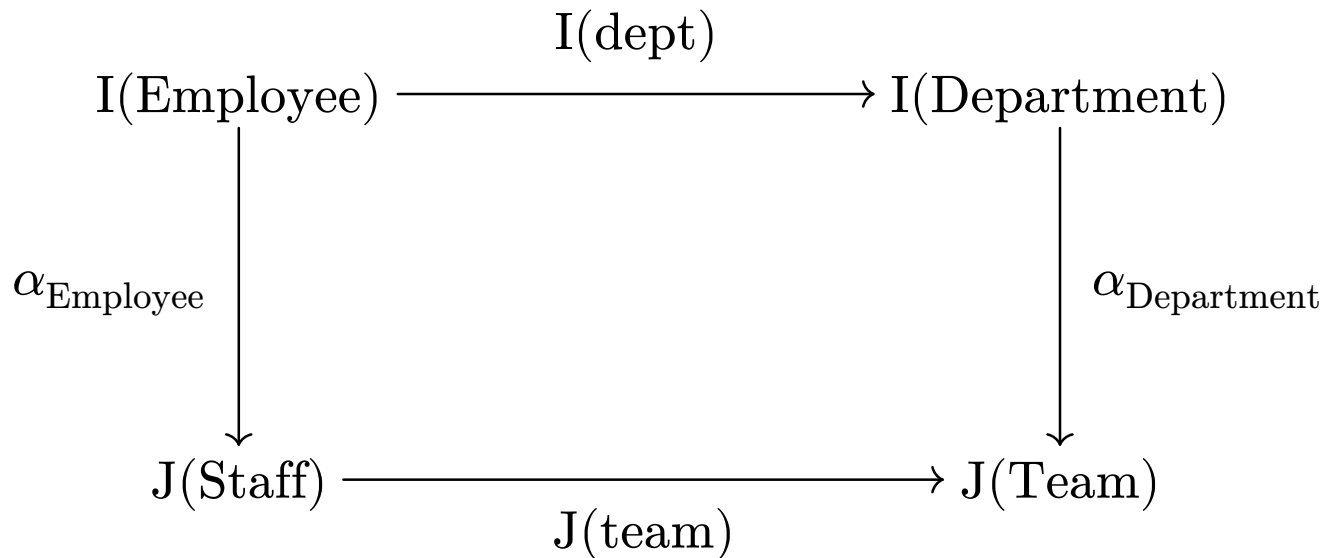


6.6. 証明②

- 射 $\text{dept} : \text{Employee} \rightarrow \text{Department}$ に対する自然条件

以下が可換図式である。

このように、自然条件 $I(\text{dept}); \alpha_{\text{Department}} = \alpha_{\text{Employee}}; J(\text{team})$ が成立していることがわかる。

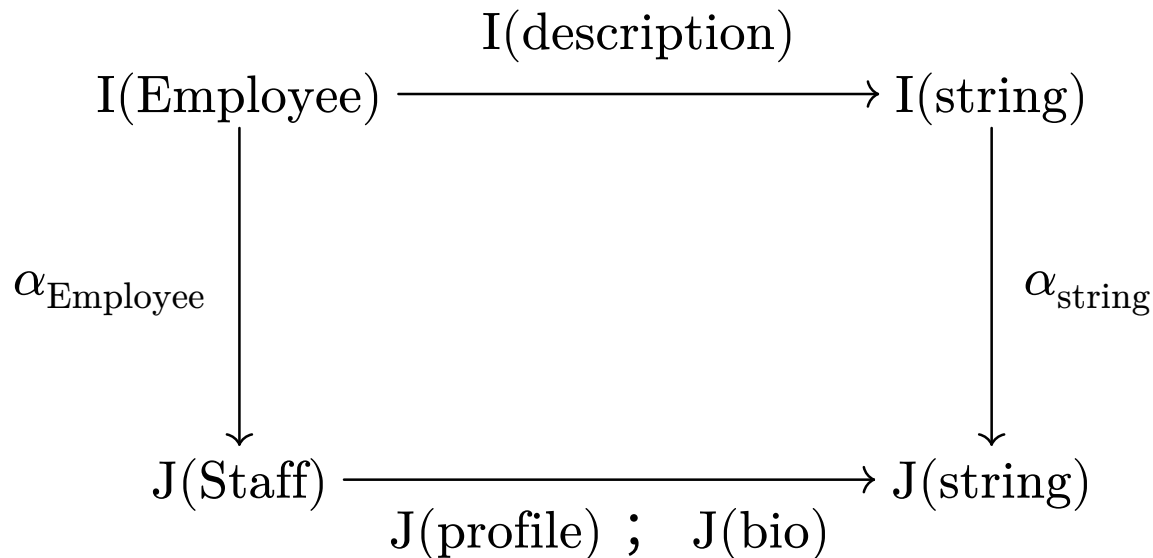


6.6. 証明②

- 射 $\text{description} : \text{Employee} \rightarrow \text{string}$ に対する自然条件

以下が可換図式である。

このように、自然条件 $I(\text{description}); \alpha_{\text{string}} = \alpha_{\text{Employee}}; (J(\text{profile}); J(\text{bio}))$ が成立していることがわかる。

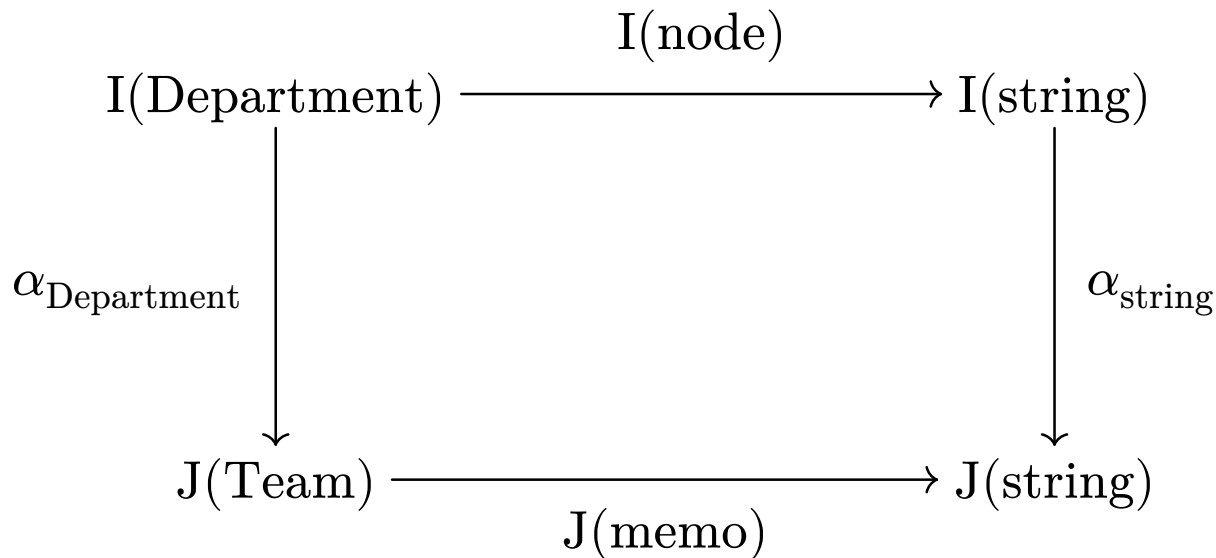


6.6. 証明②

- 射 $\text{note} : \text{Department} \rightarrow \text{string}$ に対する自然条件

以下が可換図式である。

このように、自然条件 $I(\text{node}); \alpha_{\text{string}} = \alpha_{\text{Department}}; J(\text{memo})$ が成立していることがわかる。



6.6. 証明②

従って、自然変換 $\alpha : I \Rightarrow F; J$ が存在する。

7. 6.で目的の自然変換ができたので、自然変換 α の存在により、移行後のインスタンス J は、移行元インスタンス I を関手 F に沿って整合的に受け止めていることが示された。

(移行関手という Kan 拡張の理解をすると理解できるであろう関手を用いて、実際のマイグレーションの操作を行えそうだという理解をしています。しかし、現状はマイグレーション後の結果についてのデータ整合の証明しかできていません。)